

An Empirical Analysis of Entangled Enhancement in Autonomous Vehicle Model Pipeline

Rashedul Arefin Ifty, Fumio Machida

University of Tsukuba

Tsukuba, Japan

ifty.rashedul@sd.cs.tsukuba.ac.jp, machida@cs.tsukuba.ac.jp

Abstract—Autonomous Vehicles (AVs) are increasingly built as a pipeline of separately trained perception, prediction, and planning models. Because the models are trained separately, each one is usually updated on its own while the others stay fixed. Retraining a single upstream model can nonetheless degrade the models downstream of it, because those models were built and evaluated on the output of the earlier version. This failure is known as entangled enhancement, where an upstream model improves on its own metric while the system as a whole degrades. It has so far been studied analytically in two-model systems, and has not been investigated across an AV pipeline. In this paper, we first formally define entangled enhancement in the perception–prediction–planning pipeline, separating a model’s own error from the error it inherits from upstream by measuring every downstream model both on ground-truth inputs and on the live output of its neighbour. We then measure it on a three-model pipeline built from the nuScenes dataset, retraining the perception model across a geographic domain shift and recording the effect on the prediction and planning models. Across 15 retraining updates that vary the fine-tuning data size, the number of epochs, and the random seed, the perception model improves every time, and in 6 of the updates the L2 displacement error of the planning model rises at the same time. Repeating the campaign on a class-balanced training split gives the same outcome in 4 of a further 12 updates. The two planning metrics also diverge. On one update the collision rate falls to zero while L2 agreement worsens, so no single metric on the system output is sufficient to admit an update.

Index Terms—autonomous driving, cascade effects, machine learning systems, model pipelines, regression testing, retraining, software reliability

I. INTRODUCTION

Autonomous Vehicles (AVs) are increasingly built as a pipeline of separately trained perception, prediction, and planning models. Perception detects the surrounding agents, prediction forecasts their motion, and planning selects the driving trajectory [1], [2]. Because the models are trained separately, each one can be replaced without retraining the others. In deployment, such a pipeline is maintained under domain drift one model at a time. A perception model is retrained for a new city, a prediction model is updated when traffic changes, and a planning model is revised for new regulations.

In practice, one model is usually retrained while the others stay fixed. The retrained model then becomes better on its own metric, while the models after it become worse. The reason is that those models were built and evaluated on the output of the earlier version of the retrained model.

The first reported instance of this self-defeating behavior appeared in an object-detection setting [3], where a better perception model reduced the accuracy of the downstream task. Wang and Machida [4] modeled and analyzed entangled enhancement in a two-model machine learning system by a continuous-time Markov chain. Entangled enhancement has also been reported in a sentence-classification system [5]. Sculley et al. [6] describe the same difficulty more generally. When learning components depend on each other, changing one of them creates maintenance problems for the others. However, no existing study examines entangled enhancement across an entire perception–prediction–planning pipeline for AVs. Although previous studies also consider the perception module of an AV, their analysis was limited to two sequential models and focused only on the perception component.

This failure mode is specific to pipelines of separately trained models. An end-to-end network is trained as a single model, so its components are not updated independently and no interface exists between them to shift. To address this gap, this paper studies how a single-model retraining event propagates through an AV pipeline, using empirical analysis. For the empirical analysis, we build a three-model pipeline on the nuScenes dataset [7]. The perception model is YOLOv11n [8], an object detection model applied to the front-camera image. The prediction model is a constant-velocity trajectory model. The planning model is a rule-based planner from the Intelligent Driver Model (IDM) family [9], whose proposal-and-score design follows PDM-Closed [10]. We train the perception model on Boston scenes and fine-tune it on Singapore scenes, a well-known domain gap within nuScenes, and investigate how the updates affect the downstream performance. To separate a model’s own error from the error it inherits from upstream, each downstream model is measured in two modes, one with ground-truth inputs and one with the live upstream output. Because a single retraining is only one data point, we run a campaign of 15 retraining updates that vary the amount of fine-tuning data, the number of training epochs, and the random seed.

This paper makes two contributions.

- We formally define entangled enhancement in the perception–prediction–planning pipeline for AVs, and introduce the cascade coupling factor that quantifies the intensity of entangled enhancement.

- We empirically investigate entangled enhancement in an AV pipeline built from the nuScenes dataset, showing that six out of 15 retraining updates exhibit it.

The remainder of the paper is organized as follows. Section II formalizes the pipeline model and the definition of entangled enhancement. Section III details the experimental setup and the retraining campaign. Section IV reports the empirical evidence of entangled enhancement. Section V discusses the mechanism and its implications, Section VI reviews related work, and Section VII concludes.

II. SYSTEM MODEL AND METHODOLOGY

A. Formal Pipeline Model

We model an AV pipeline as a composition of three functions,

$$S = C_1 \circ C_2 \circ C_3, \quad S(x) = C_3(C_2(C_1(x))), \quad (1)$$

where x is an input camera frame processed through the three models:

- **Perception** (C_1). The output $z_1 = C_1(x)$ is a list of the objects the model found in the image. For each object it gives the type, the size, and the position on the road relative to the ego vehicle.
- **Prediction** (C_2). The output $z_2 = C_2(z_1)$ is, for every detected object, a forecast trajectory over the next several seconds.
- **Planning** (C_3). The output $z_3 = C_3(z_2)$ is the ego vehicle's planned trajectory, a sequence of positions over a 3 s horizon.

The outputs z_1 and z_2 connect one model to the next. A change in C_1 changes z_1 , which is the input of C_2 . The change then reaches z_2 , which is the input of C_3 . Now suppose that only C_1 is retrained. The parameters of C_2 and C_3 are unchanged. Any change in their outputs therefore comes only from their inputs.

B. Entangled Enhancement

Entangled enhancement occurs when an upstream model is retrained and improves its own performance, and yet the system as a whole degrades [4]. This entanglement happens because the downstream models were tuned to the statistics of the *original* upstream output and are shifted after the upstream model update. We denote by δ_1 the improvement of the perception performance of C_1 after retraining, and by Δ_3 the degradation of the planning performance of C_3 . Entangled enhancement is formally specified by the joint condition

$$\underbrace{\delta_1 > 0}_{C_1 \text{ improves}} \quad \wedge \quad \underbrace{\Delta_3 > 0}_{\text{system degrades}}. \quad (2)$$

The condition requires the upstream model to improve on its *own* performance ($\delta_1 > 0$) while the system nonetheless degrades ($\Delta_3 > 0$). The upstream model must improve for this to be entangled enhancement. If the upstream model became worse and the system became worse with it, that is an ordinary regression.

C. Two-Mode Measurement and the Isolation Property

To make entangled enhancement measurable, we separate a model's own error from the error inherited from the upstream models. Let z_k denote the output of model C_k , and let $M_k(z_k)$ be an error metric evaluated on that output, with lower values better. We evaluate each downstream model in two modes. In *isolated* mode a model receives ground-truth inputs, and its output is written z_k^{iso} . For C_2 and C_3 ,

$$z_2^{\text{iso}} = C_2(z_1^*), \quad z_3^{\text{iso}} = C_3(z_2^*), \quad (3)$$

where z_k^* is the ground-truth value of C_k . In *pipeline* mode a model receives the direct output of the upstream neighbor, and its output is written z_k^{pipe} ,

$$z_2^{\text{pipe}} = C_2(C_1(x)), \quad z_3^{\text{pipe}} = C_3(C_2(C_1(x))). \quad (4)$$

The metric in isolated mode, $M_k(z_k^{\text{iso}})$, reflects the model's own error, while the metric in pipeline mode, $M_k(z_k^{\text{pipe}})$, reflects its own error together with the error inherited from the upstream model. The difference between the two is the inherited part. Isolated mode needs ground-truth inputs, so it can be run only in an experiment, not in a real vehicle. It shows whether a change came from the model itself or from its input.

Consider a retraining event that replaces an upstream model C_k by an updated model C'_k , whose outputs we write z'_k . If the pipeline is properly composed, the retraining leaves the isolated-mode output of every downstream model that was not retrained unchanged, because that output depends only on the ground-truth inputs. We call this the *isolation property*,

$$M_k(z_k^{\text{iso}}) = M_k(z_k^{\text{iso}'}). \quad (5)$$

Equation (5) is a testable property of the setup rather than an assumption. Any change in the isolated-mode metric of a model that was not retrained indicates contamination, and we verify the equation in Section IV. The effect that the retraining causes at model C_k is the change in its pipeline metric,

$$\Delta_k = M_k(z_k^{\text{pipe}'}) - M_k(z_k^{\text{pipe}}), \quad (6)$$

which, given (5), is entirely inherited from the upstream update.

D. Coupling Factor

To quantify the intensity of entangled enhancement, we relate the system performance degradation Δ_3 to the upstream improvement δ_1 , through the *cascade coupling factor*

$$\rho_{1 \rightarrow 3} = \frac{\Delta_3}{\delta_1}. \quad (7)$$

Under the sign convention above, a positive δ_1 improves the upstream model and a positive Δ_3 worsens the system. Entangled enhancement therefore corresponds to $\rho_{1 \rightarrow 3} > 0$, and the magnitude of $\rho_{1 \rightarrow 3}$ measures how much regression of the planning metric accompanies a unit of upstream improvement. The factor is defined whenever $\delta_1 \neq 0$, which holds for every update reported in this paper, and it can be computed from the δ_1 and Δ_3 columns of Table II.

III. EXPERIMENTAL SETUP

The objective of this section is to construct a pipeline in which a single retraining event can be applied to C_1 and its downstream effect measured under exact controls. We describe the dataset and the three models, the retraining procedure, the metrics recorded at each model, and the campaign of 15 updates over which they are measured.

A. Dataset and Components

We use the full nuScenes v1.0 trainval split [7]. A nuScenes *scene* is a ≈ 20 s driving clip, annotated at 2Hz. Each of these annotated samples is a *keyframe*. A scene contributes about 40 keyframes, and we use the front-camera (CAM_FRONT) keyframe as one evaluation frame throughout. The trainval split contains 850 scenes (34,149 keyframes in total, ≈ 45 GB), of which 467 scenes were recorded in Boston and 383 in Singapore. Partitioning by recorded location and splitting each city 60/40 into train and validation gives `boston_train/boston_val` (280/187 scenes) and `singapore_train/singapore_val` (229/154 scenes). The Boston perception model is trained on the 11,087 CAM_FRONT keyframes of `boston_train`. All measurements use a fixed subset of `singapore_val`, obtained by taking every third scene of its 154 scenes to give 50 scenes, comprising $n = 2,020$ annotated keyframes. The subset keeps the evaluation affordable across the 15 fine-tunes of the campaign while still spanning the whole city. This same 50-scene subset is reused, unchanged, for every one of the 15 updates, which keeps the comparisons exact. For C_1 training, the 3D box annotations are projected onto the front-camera keyframes to produce 2D labels over the eight driving classes that the pipeline consumes, namely car, truck, bus, trailer, construction vehicle, motorcycle, bicycle, and pedestrian.

The perception model C_1 is YOLOv11n [8], an object detection model applied to the front-camera image. We fine-tune it for 10 epochs on `boston_train` from COCO weights to obtain the original model. To generate an updated model, the same YOLOv11n is fine-tuned further on `singapore_train` starting from the Boston checkpoint. At inference, each 2D detection is lifted to an ego-frame ground position by intersecting its bottom-center pixel ray with the ground plane.

The prediction model C_2 is a constant-velocity motion model, which is deterministic and carries no trainable parameters. It forecasts an agent by projecting it forward at whatever velocity it is given. In isolated mode it is given the ground-truth velocity of each agent. In pipeline mode it receives the detections of C_1 on a single frame, which carry position but no velocity, so the projection is made at zero velocity and the forecast holds each agent at its detected position. Any change in its pipeline output is therefore attributable solely to a change in C_1 .

The planning model C_3 is a rule-based planner of the proposal-and-score type, built on the Intelligent Driver Model [9]. It works in two stages. In the proposal stage it

enumerates a finite set $\mathcal{P}$ of candidate trajectories, obtained by sweeping target speeds and lateral offsets around the current lane. In the scoring stage it selects the candidate π^* of minimum cost,

$$\pi^* = \arg \min_{\pi \in \mathcal{P}} J_{\text{prog}}(\pi) + \lambda \sum_{a \in z_2} \phi(d(\pi, a)). \quad (8)$$

The cost has two terms. The progress term $J_{\text{prog}}(\pi)$ penalises a candidate for advancing slowly along the route. The safety term penalises a candidate for passing close to any predicted agent. For an agent a in the predicted futures z_2 , $d(\pi, a)$ is the closest distance between candidate π and that agent over the next 3 seconds, and the penalty ϕ is a decreasing function of this distance. A candidate that passes near an agent therefore receives a large penalty. The weight λ sets how strongly safety is traded against progress. This design reproduces the PDM-Closed [10] architecture without the learned scorer. The safety term is the only place where C_3 reads z_2 . A detection that moves, appears, or disappears changes the predicted futures, which changes the safety term, which changes the selected candidate. This is the path by which a retraining of C_1 reaches the plan.

B. Retraining Procedure

The source domain is Boston and the target domain is Singapore. The shift between them is a well-studied domain gap within nuScenes, with left-hand versus right-hand traffic, different vehicle and agent mixes, and different lighting conditions [7]. Before the experiments begin, C_1 is trained on the source domain to obtain the original C_1 , and C_2 and C_3 are frozen for the remainder of the study.

Each retraining event consists of three steps, in which C_1 is the only model that changes.

- 1) The original pipeline is measured on `singapore_val`, recording the detection quality of C_1 together with the metrics of C_2 and C_3 in isolated and pipeline mode.
- 2) C_1 is fine-tuned on the target domain to obtain the updated model C'_1 .
- 3) The updated pipeline is measured on the same frames. Comparing the two records verifies the isolation property (5) and gives the changes (6) at C_2 and C_3 .

C. Metrics

The perception model C_1 is scored with standard detection metrics (mAP@50, mAP@50-95, precision, recall) on the target-domain validation images.

The prediction model C_2 is scored with the minimum average displacement error (minADE), the standard metric for predictors that may output several candidate futures, or modes, for one agent. For an agent with forecast positions $\hat{p}_t^{(m)}$ under mode m and ground-truth positions p_t^* over T future steps,

$$\text{minADE} = \min_m \frac{1}{T} \sum_{t=1}^T \|\hat{p}_t^{(m)} - p_t^*\|_2, \quad (9)$$

averaged over agents. The prediction model used here is deterministic and has a single mode, so the minimum is taken over that one mode.

The planning model C_3 is scored with the L2 displacement error between the planned ego trajectory $\hat{e}_t$ and the trajectory the human driver actually took, e_t^* , measured τ seconds ahead of the current frame,

$$\text{L2@}\tau = \|\hat{e}_\tau - e_\tau^*\|_2, \quad \tau \in \{1, 2, 3\} \text{ s.} \quad (10)$$

The collision rate is reported alongside it.

The *plan shift* is recorded for the campaign. It is the distance between the plan produced with the original C_1 and the plan produced with the updated C_1' on the same scene. It measures how far a retraining moves the driving decision, without reference to the human path.

L2 is the standard open-loop planning protocol of nuScenes, and earlier studies on end-to-end driving models use it as well [2], [11], [12]. The protocol has known limitations. We use it differentially, comparing two runs that differ only in C_1 , and Sections IV and VI discuss what this comparison does and does not show.

D. The Retraining Campaign

We run a campaign of 15 **retraining updates** of C_1 , each measured through the three steps of Section III-B. The campaign varies three factors.

- **Fine-tuning data size.** We fine-tune C_1 on three amounts of Singapore data, namely the full `singapore_train` set of 229 scenes, one half of it, and one quarter of it. The half and the quarter are obtained by taking every second and every fourth scene of a fixed scene ordering, which makes the same subsets available to every run. A smaller set gives C_1 less data for adapting to the new city. Varying this amount therefore tests whether the downstream effect depends on how much C_1 was able to adapt.
- **Training length.** We use 5, 10, and 20 epochs, to test whether longer fine-tuning changes the downstream effect.
- **Random seed.** We repeat each setting with three seeds, which separates a result from seed noise.

These three factors are combined into three data sizes at 10 epochs, plus two additional training lengths (5 and 20) at the full data size, each repeated over three seeds, giving $3 \times 3 + 2 \times 3 = 15$ updates. Only the Singapore fine-tuned C_1' differs between the updates. The Boston-trained C_1 and the frozen C_2 and C_3 are shared across all 15. Within each update the fine-tuning run uses a single fixed random seed. The evaluation of C_2 and C_3 is deterministic, because neither model has trainable parameters. Repeating a measurement reproduces the same numbers exactly. The isolation property (5) can be verified by checking that the isolated-mode metrics are identical in the two runs, and any difference at all, however small, indicates a fault in the measurement.

E. Measurement Protocol

For each of the 15 updates we record the metrics of Section III-C on `singapore_val`, once with C_1 in place and once with C_1' in place. From the two records we compute δ_1 , Δ_2 , Δ_3 , the plan shift, and whether the update meets the condition for entangled enhancement (Section II-B).

IV. EVALUATION RESULTS

This section reports whether entangled enhancement occurs in the AV pipeline.

We begin with a single retraining event and ask whether entangled enhancement occurs in it (Section IV-A). A positive answer raises two doubts that must be settled before the result can be believed. The planning model may have degraded because the retrained C_1 was worse, which would make the case an ordinary regression, so we check the perception model (Section IV-B). The change may also come from the measurement setup rather than from the update, so we check that the models we did not retrain were untouched (Section IV-C). With both doubts settled, we locate the point along the pipeline where the retraining has its largest effect (Section IV-D).

A single event cannot show how often the condition arises. We therefore repeat the measurement over the 15 updates of the campaign (Section IV-E). One explanation remains available even then, namely that the effect is an artifact of the class imbalance in the training data, and the final experiment removes that imbalance and repeats the campaign over a further 12 updates (Section IV-F). Section IV-G reports the training diagnostics that the preceding results rely on.

A. The Reference Update

A single retraining event is enough to test whether the joint condition (2) can be met in practice. The reference update is the full-data, 10-epoch, seed-1 fine-tune, and Table I reports its measurements on `singapore_val` ($n = 2,020$ frames).

The perception model improves on its own metric, with $\delta_1 = +0.119$ as mAP@50 rises from 0.137 to 0.256. The input it gives to C_2 improves as well, by $\Delta_2 = -2.17\text{m}$, a 27% reduction. C_3 nonetheless degrades. Its L2 error against the human trajectory rises from 5.55 to 5.67m, so $\Delta_3 = +0.124\text{m}$. Both clauses of (2) are satisfied, and the update is an instance of entangled enhancement in an AV pipeline. Sculley et al. [6] describe this kind of failure qualitatively; the measurements here quantify one case of it under controlled conditions. The degradation originates in the input rather than in C_3 itself. In isolated mode the L2 error of C_3 is 4.64m in both runs, because the planning model receives identical ground-truth inputs. Only the pipeline-mode value moves, and Fig. 1 shows the two pairs.

The two metrics of C_3 move in opposite directions. The collision rate falls from 0.020 to zero over the same update in which the L2 error rises. A single-metric check on the system output would have reached opposite verdicts depending on which metric it read.

TABLE I: Reference update measured on Singapore validation ($n = 2,020$ frames). “Before” is the Boston-trained C_1 and “after” is the Singapore-fine-tuned C'_1 .

Metric	Before	After	Δ	$\Delta\%$
C_1 mAP@50	0.1370	0.2556	+0.1186	+86.6
C_2 minADE, iso. (m)	1.1211	1.1211	0.0000	0.00
C_2 minADE, pipe. (m)	7.9310	5.7590	-2.1720	-27.4
C_3 L2@3s, iso. (m)	4.6384	4.6384	0.0000	0.00
C_3 L2@3s, pipe. (m)	5.5463	5.6703	+0.1240	+2.2
C_3 collision rate, pipe.	0.020	0.000	-0.0200	n/a

B. Perception Quality After Retraining

If the retrained C_1 were weaker than the original, the planning degradation would be an ordinary regression rather than entangled enhancement. The measurements rule this out. The mAP@50 of C_1 rises by 86.6%, and the 229-scene Singapore fine-tune set is large enough that C_1 adapts to the new city rather than over-fitting it. The planning degradation therefore cannot be attributed to a weaker perception model. C_1 is not uniformly strong, however. The confusion matrix of the Boston-trained C_1 (Fig. 2) shows the dominant car class well separated, while the rarer classes are still frequently confused with background. Section IV-F examines whether this residual class imbalance is what produces the effect.

C. The Isolation Control

The measured change can be attributed to the update only if C_2 and C_3 were themselves untouched by it. Because both models are deterministic and have no trainable parameters, feeding them identical ground-truth inputs must return identical values. Any difference would mean that state or data had leaked between the two runs.

The isolated-mode rows of Table I show no such difference. The metric of C_2 is 1.1211 m in both runs, and the metric of C_3 is 0.7458, 2.4100 and 4.6384 m at 1, 2 and 3 seconds in both runs, to every recorded digit. The same equality held for all 15 updates of the campaign. The pipeline-mode changes in Table I can therefore be attributed to the retraining of C_1 , reaching C_2 and C_3 through their inputs.

D. Location of the Largest Effect

Knowing where the retraining has its largest effect indicates where a maintenance check should be placed. In Table I that effect is at the step from C_1 to C_2 . Under the Boston-trained C_1 , the metric of C_2 in pipeline mode was 7.9310 m against 1.1211 m in isolated mode, a gap of 6.81 m. After the Singapore fine-tune the pipeline-mode value fell to 5.7590 m, closing the gap to 4.64 m. This improvement of 27.39% was obtained without modifying C_2 at all.

The gap itself has two sources, because C_2 uses ground-truth velocity in isolated mode and a zero-velocity rollout in pipeline mode (Section III). It measures the detection error of C_1 together with the loss of velocity information, rather than the detection error alone. The velocity term is the same in

both runs, so the change $\Delta_2 = -2.17$ m between them is still attributable to C_1 alone.

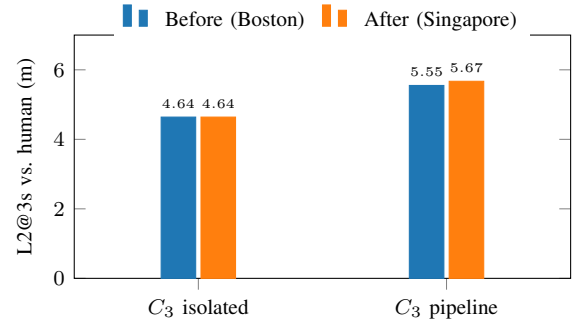


Fig. 1: Effect of the reference update on planning. The metric of C_3 in isolated mode stays fixed while its metric in pipeline mode moves.

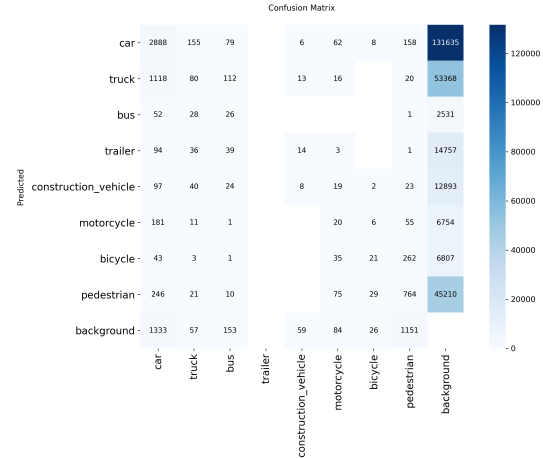


Fig. 2: Confusion matrix of the Boston-trained C_1 on full trainval.

E. Campaign Results

One update cannot show how often the condition arises, or how strongly, when the retraining configuration is varied. Table II reports all 15 updates of the campaign (Section III-D).

Fine-tuning on the new city improves the perception model every time. All 15 updates have $\delta_1 > 0$, and all 15 improve the prediction model as well, with $\Delta_2 < 0$. The planning model nonetheless degrades in 6 of those 15 updates, which meets the condition for entangled enhancement. The degradations range from +0.008 m, which is small enough that it may not exceed the run-to-run variation of the measurement, to +0.193 m. Reaching this condition required no intervention on the training data.

The remaining 9 updates improve the planning model. Nothing in the training configuration separates the two groups, so an update that improves C_1 cannot be assumed to improve the plan, and the effect has to be measured at the downstream models rather than predicted from C_1 .

TABLE II: The 15-update retraining campaign on singapore_val ($n = 2,020$ frames). A positive δ_1 marks an improved perception model, a negative Δ_2 an improved prediction, and a positive Δ_3 a worse planning model. The shift column is the plan shift in metres. “ent. enh.” marks entangled enhancement ($\delta_1 > 0$ with $\Delta_3 > 0$) and “helped” marks $\Delta_3 < 0$. Every update has $\delta_1 > 0$.

Update	data	δ_1	Δ_2	Δ_3	shift	outcome
Full, 10 ep, s1	full	+0.119	-2.17	+0.124	1.05	ent. enh.
Full, 10 ep, s2	full	+0.122	-2.28	+0.193	1.02	ent. enh.
Full, 10 ep, s3	full	+0.129	-2.15	+0.187	0.92	ent. enh.
Half, 10 ep, s1	half	+0.114	-2.55	-0.121	1.35	helped
Half, 10 ep, s2	half	+0.120	-2.71	-0.230	1.19	helped
Half, 10 ep, s3	half	+0.106	-1.77	-0.151	1.46	helped
Quarter, 10 ep, s1	qtr	+0.121	-2.71	+0.073	0.79	ent. enh.
Quarter, 10 ep, s2	qtr	+0.113	-1.29	-0.093	0.78	helped
Quarter, 10 ep, s3	qtr	+0.101	-2.82	+0.008	1.00	ent. enh.
Full, 5 ep, s1	full	+0.138	-1.08	-0.095	0.89	helped
Full, 5 ep, s2	full	+0.122	-1.88	-0.130	1.22	helped
Full, 5 ep, s3	full	+0.130	-1.08	-0.041	0.92	helped
Full, 20 ep, s1	full	+0.142	-2.59	+0.037	1.37	ent. enh.
Full, 20 ep, s2	full	+0.126	-1.77	-0.083	1.50	helped
Full, 20 ep, s3	full	+0.130	-2.44	-0.020	1.20	helped

F. Robustness to Class Imbalance

The campaign reaches the condition without any intervention on the training data. The effect could still be an artifact of the residual class imbalance visible in Fig. 2, where rare classes are confused with background.

We therefore repeat the campaign on a class-balanced training split. Images containing rare classes are oversampled so that the per-class instance distribution flattens, using a repeat factor $(n_{\max}/n_c)^p$ per class c clipped to $[1, 12]$. The validation split is left untouched, so the metrics remain comparable with those of the first campaign. This second campaign varies two balance strengths ($p \in \{0.5, 1.0\}$), two training lengths (10 and 20 epochs), and three seeds, giving a further 12 updates.

Balancing the classes does not remove the effect. The perception model improves in all 12 updates, and the planning model degrades in 4 of them. Three of those four occur at $p = 0.5$, with δ_1 up to +0.143 and Δ_3 up to +0.139 m, and one at $p = 1.0$, with $\delta_1 = +0.125$ and $\Delta_3 = +0.075$ m. Cases appear at both balance strengths and at both training lengths. The residual class imbalance is therefore not the source of the effect.

G. Training Diagnostics

The results above assume that each fine-tune actually converged, and that the C_1 was not left under-trained or over-fitted. The training curves confirm this. Both the Boston and the Singapore fine-tunes converged within their epoch budgets, and Fig. 3 shows the Boston run. The box, classification and distribution-focal losses decrease on both the training and the validation split, while precision, recall and mAP@50 increase,

with mAP@50 rising from 0.116 at epoch 1 to 0.137 at epoch 10.

The cost of one update is what makes a campaign of this size possible. A single update, comprising one fine-tune and the two evaluations, takes about 24 GPU-minutes, so the 15-update campaign ran in about six GPU-hours. Training an end-to-end planner requires about 144 GPU-hours [13], at which cost a campaign of repeated retraining is not practical.

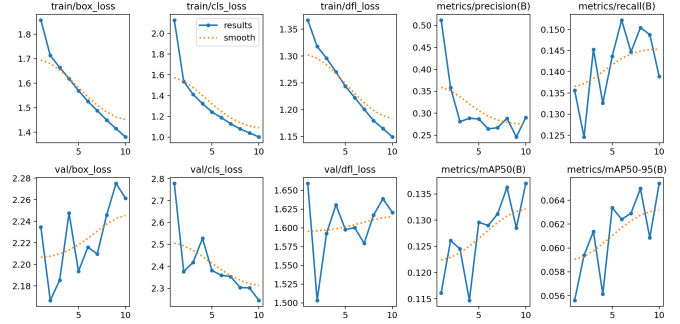


Fig. 3: YOLOv11n C_1 Boston fine-tune training curves.

V. DISCUSSION

The measurements show that an improved perception model can degrade the plan, but not why. This section explains the mechanism and draws its implications for maintenance.

A. Mechanism of the Cascade

Under the Boston-trained C_1 , the forecasts reaching C_3 carried 6.81 m more error than the same model produces on ground-truth inputs. At that size of error, agents were missed altogether or placed far from their true positions, so the safety term of (8) was rarely triggered. C_3 then reverted to its own speed and lane rules, and on these scenes those rules happened to produce trajectories close to the one the human driver took.

The updated C_1 placed detections near their true positions, which activated the safety term. C_3 began to react to agents it had previously not seen, and its trajectories moved away from the human path. C_3 had not changed; what changed was how often its safety term was active. Two consequences follow. A downstream model can absorb upstream error, so its measured quality understates how much it depends on the upstream input. An upstream improvement can then expose that dependence. An updated model therefore cannot be admitted on the basis of its own output alone.

The same mechanism cautions against reading the planning metric too literally. The collision rate fell from 0.020 to zero over the reference update, while L2 rose. The higher L2 arises because the improved C_1 caused C_3 to react to real agents instead of following speed and lane rules that happened to track the recorded human path. This is a known weakness of open-loop L2 on nuScenes: prediction that ignores perception entirely remains competitive [12], and open-loop error is weakly tied to closed-loop driving quality [14]. A worse score on this metric can accompany a safer planning model. The

planning results here should be read as reduced agreement with the human trajectory rather than as an established safety regression, and a closed-loop or NAVSIM-style metric [15] would be the appropriate check.

B. Implications for Maintenance

The results give three rules for admitting an update. First, an update must not be admitted on the updated model’s own metric, because every update in the campaign improved that metric. Second, admission must not stop at the next model either, because the entangled-enhancement cases are exactly those in which C_1 improved and C_2 improved while C_3 degraded. Third, a maintenance policy should assess several candidate updates rather than a single one, because the effect on planning varies across updates. Identifying the condition needs both ends of the pipeline. A change in the planning metric alone does not identify it, because $\Delta_3 > 0$ also describes an ordinary regression in which C_1 became worse, so the perception metric δ_1 has to be read alongside it.

A cheaper check at the upstream interface does not substitute for the downstream measurement. To test this, we compared the detections of the updated C_1 against those of the original C_1 on the same scenes, for each of the 15 updates, using two-sample tests on the per-image count, the confidence, the box-center position, and the class histogram. The resulting score tracked how far the plan moved, with a rank correlation of ≈ 0.56 , but it did not separate the updates that worsened the planning metric from the rest. Its area under the receiver operating characteristic curve was ≈ 0.48 , where 0.5 is the value expected from random ordering. Every retrained C_1 improved, so the scores fall in a narrow range in which the two groups overlap. How much the output of C_1 moves therefore indicates the size of the downstream disturbance but not its direction, and the effect at C_3 has to be measured rather than predicted from upstream.

VI. RELATED WORK

A. End-to-End Driving and Maintenance Limits

UniAD put detection, tracking, mapping, motion forecasting, occupancy prediction, and planning into a single planning-oriented network [2]. VAD replaced dense rasterized scene representations with vectorized ones for efficiency [11]. SparseDrive introduced fully sparse scene representations and reported the resource profile of the flagship systems: $7.2\times$ faster training and $5\times$ faster inference than UniAD, whose training takes about 144 GPU-hours [13]. The main survey of the field lists interpretability, causal confusion, and robustness under distribution shift as open challenges, and it does not treat component-level maintenance or retraining policy at all [1].

A recent end-to-end repair literature addresses maintenance directly. CorrectAD builds a self-correction loop that diagnoses failures, generates targeted training video with a diffusion model, and fine-tunes the entire end-to-end planner on old plus new data, correcting 62.5% of failure cases on nuScenes [16]. R2SE avoids full-model retraining by freezing

the generalist model and training reinforcement-learned low-rank adapter specialists on failure regions, motivated by the catastrophic forgetting that naive fine-tuning causes [17]. Both reintroduce pipeline structure to make maintenance tractable, which motivates the pipeline architecture adopted in this work.

B. Evaluation of Driving Systems

Our planning metric is open-loop displacement error on nuScenes, and the limits of this protocol are well documented. Codevilla et al. showed as early as 2018 that offline prediction error correlates only weakly with actual closed-loop driving quality [14]. Zhai et al. found that a trivial MLP using only ego state, with no perception at all, matches perception-based end-to-end methods on nuScenes open-loop L2 [12]. Dauner et al. showed that open-loop ego-forecasting and closed-loop driving are misaligned tasks, and that a largely rule-based planner, PDM-Closed, outperformed learned planners [10]. Their later NAVSIM benchmark provides short-horizon non-reactive metrics that align far better with closed-loop performance [15]. These critiques target open-loop metrics used to rank architectures. Our use is differential, comparing two runs with the same planning model, the same scenes and one upstream change, so a bias common to both runs cancels. We report the collision rate alongside L2 and return to the caveat in Section V.

C. Model Updates, Regression, and Interface Compatibility

What we measure at system scale has per-sample analogues in the model-update literature. Yan et al. showed that model updates cause negative flips, samples the old model handled correctly that the new model gets wrong, at rates of 4–5% even between equal-accuracy retrains of identical architectures, and they proposed positive-congruent training with focal distillation to suppress them [18]. The backward-compatible training of Shen et al. constrains a new upstream representation model so that unchanged downstream consumers keep working [19], which is the same situation as updating C_1 beneath an unchanged C_2 and C_3 . On the monitoring side, the data validation system of Breck et al. runs two-sample distribution tests between pipeline stages in production [20]. On a nuScenes detector–tracker–predictor stack of a similar structure to ours, Ivanovic et al. showed that propagating state uncertainty across the perception–prediction interface, instead of passing point estimates, improves forecasting accuracy [21]. These works provide mitigation techniques. This paper contributes the complementary controlled measurement of the cascade that such techniques are intended to mitigate.

D. Reliability Modeling of Machine Learning Systems

The technical-debt analysis of Sculley et al. introduced the principle that changing anything changes everything, together with unstable data dependencies, and correction cascades as qualitative failure modes of ML systems [6]. The base paper for this work, Wang and Machida [4], formalized imperfect retraining of a two-model machine learning system as a continuous-time Markov chain, named entangled

enhancement, and compared progressive and conservative maintenance policies. Their follow-on work casts the same problem as an availability–continuity trade-off [5]. Both are analytical models whose state space grows as 2^N for N components, and both are calibrated on synthetic transition rates. This paper provides an empirical counterpart. The isolation property we verify empirically is the property their state factorization assumes, and the coupling factor we propose (Section II-D) is a measured statistic that can replace those synthetic rates.

VII. CONCLUSION

We presented a controlled study of single-model retraining in a perception–prediction–planning pipeline under a geographic domain shift. The two-mode measurement, with a verified isolation control, was applied on the full nuScenes trainval split (2,020 evaluation frames) across a campaign of 15 retraining updates rather than a single event.

Retraining C_1 disturbed the downstream plan in every case. The perception model improved on its own metric in all 15 updates, and in 6 of them the L2 error of the planning model rose at the same time. Repeating the campaign on a class-balanced training split gave the same outcome in 4 of a further 12 updates, so the effect does not depend on the class imbalance of the original data. Entangled enhancement has so far been studied analytically, and these measurements show that it is also observable in an AV pipeline.

The two planning metrics moreover diverge. On the reference update the collision rate improved to zero while L2 agreement worsened. Each of the six cases would have passed a review of C_1 alone, and also a review of the collision rate alone. Neither model-level validation nor any single-metric check on the system output is sufficient for safe maintenance. Measuring a cascade under exact controls requires an architecture that exposes the connections between its models, which is what the pipeline setting provides.

REFERENCES

- [1] L. Chen, P. Wu, K. Chitta, B. Jaeger, A. Geiger, and H. Li, “End-to-end autonomous driving: Challenges and frontiers,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [2] Y. Hu, J. Yang, L. Chen, K. Li, C. Sima, X. Zhu, S. Chai, S. Du, T. Lin, W. Wang, L. Lu, X. Jia, Q. Liu, J. Dai, Y. Qiao, and H. Li, “Planning-oriented autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.
- [3] R. Wu, C. Guo, A. Hannun, and L. van der Maaten, “Fixes that fail: Self-defeating improvements in machine-learning systems,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 34, 2021, pp. 11 745–11 756.
- [4] Z. Wang and F. Machida, “Maintaining performance of a machine learning system against imperfect retraining,” in *2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC)*. IEEE, 2024, pp. 1782–1787.
- [5] Z. Wang and F. Machida, “Exploiting the availability-continuity trade-off in imperfect retraining of machine learning systems,” in *2025 IEEE 36th International Symposium on Software Reliability Engineering (ISSRE)*, 2025, pp. 406–417.
- [6] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, “Hidden technical debt in machine learning systems,” in *Advances in Neural Information Processing Systems (NIPS)*, vol. 28, 2015, pp. 2503–2511.
- [7] H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom, “nusenes: A multimodal dataset for autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11 621–11 631.
- [8] G. Jocher and J. Qiu, “Ultralytics yolo11,” <https://github.com/ultralytics/ultralytics>, 2024.
- [9] M. Treiber, A. Hennecke, and D. Helbing, “Congested traffic states in empirical observations and microscopic simulations,” *Physical Review E*, vol. 62, no. 2, pp. 1805–1824, 2000.
- [10] D. Dauner, M. Hallgarten, A. Geiger, and K. Chitta, “Parting with misconceptions about learning-based vehicle motion planning,” in *Conference on Robot Learning (CoRL)*, 2023.
- [11] B. Jiang, S. Chen, Q. Xu, B. Liao, J. Chen, H. Zhou, Q. Zhang, W. Liu, C. Huang, and X. Wang, “Vad: Vectorized scene representation for efficient autonomous driving,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [12] J.-T. Zhai, Z. Feng, J. Du, Y. Mao, J.-J. Liu, Z. Tan, Y. Zhang, X. Ye, and J. Wang, “Rethinking the open-loop evaluation of end-to-end autonomous driving in nusenes,” *arXiv preprint arXiv:2305.10430*, 2023.
- [13] W. Sun, X. Lin, Y. Shi, C. Zhang, H. Wu, and S. Zheng, “Sparsedrive: End-to-end autonomous driving via sparse scene representation,” *arXiv preprint arXiv:2405.19620*, 2024.
- [14] F. Codevilla, A. M. López, V. Koltun, and A. Dosovitskiy, “On offline evaluation of vision-based driving models,” in *European Conference on Computer Vision (ECCV)*, 2018.
- [15] D. Dauner, M. Hallgarten, T. Li, X. Weng, Z. Huang, Z. Yang, H. Li, I. Gilitschenski, B. Ivanovic, M. Pavone, A. Geiger, and K. Chitta, “Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024.
- [16] E. Ma, L. Zhou, T. Tang, J. Zhang, J. Jiang, Z. Zhang, D. Han, K. Zhan, X. Zhang, X. Lang, H. Sun, X. Zhou, D. Lin, and K. Yu, “Correctad: A self-correcting agentic system to improve end-to-end planning in autonomous driving,” *arXiv preprint arXiv:2511.13297*, 2025.
- [17] H. Liu, T. Li, H. Yang, L. Chen, C. Wang, K. Guo, H. Tian, H. Li, H. Li, and C. Lv, “Reinforced refinement with self-aware expansion for end-to-end autonomous driving,” *arXiv preprint arXiv:2506.09800*, 2025.
- [18] S. Yan, Y. Xiong, K. Kundu, S. Yang, S. Deng, M. Wang, W. Xia, and S. Soatto, “Positive-congruent training: Towards regression-free model updates,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 14 299–14 308.
- [19] Y. Shen, Y. Xiong, W. Xia, and S. Soatto, “Towards backward-compatible representation learning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 6368–6377.
- [20] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, “Data validation for machine learning,” in *Proceedings of Machine Learning and Systems (SysML/MLSys)*, vol. 1, 2019.
- [21] B. Ivanovic, Y. Lin, S. Shrivastava, P. Chakravarty, and M. Pavone, “Propagating state uncertainty through trajectory forecasting,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.